

Contents

1 Making Web Requests with HTTP *3*

1.1 HTTP Requests *3*

1.2 Using HTTP with Web-Based APIs *4*

1.3 HTTP Responses *5*

1.4 Data Formats in Responses *5*

2 Using and Creating APIs *7*

2.1 Setting up an API *8*

2.1.1 What is an endpoint? *9*

2.1.2 Reading all resources *10*

2.2 Requests with cURL *11*

2.3 Updating and Deleting Resources with cURL *12*

2.3.1 Fully replacing a resource with PUT *12*

2.3.2 Partially updating a resource with PATCH *13*

2.3.3 Removing a resource with DELETE *14*

2.3.4 Headers with `-i` and `-v` *14*

2.4 Authentication and API Keys *15*

2.5 Creating resources with POST *17*

2.6 Postman *19*

2.6.1 Building your first request in Postman *19*

2.6.2 Why this matters once requests get complicated *20*

2.7 Summary *20*

3 Data Compression and Decompression *23*

3.1 Data Compression and Decompression 23
3.2 Entropy 23
3.3 Entropy and Compression 23

4 Dealing with JSON and XML 25

A Answers to Exercises 27

Index 29

Chapter 1

Making Web Requests with HTTP

The Hypertext Transfer Protocol (HTTP) is the protocol used for transmitting hypertext over the World Wide Web. It is the foundation of any data exchange on the web, and it is a protocol used for transmitting hypertext requests from clients (like a user's browser) to servers, which respond with the requested resources.

1.1 HTTP Requests

An HTTP request is made up of several components:

- **Method**: The HTTP method, like GET (retrieve data), POST (send data), PUT (update data), DELETE (remove data), and so on.
- **URL**: The URL of the resource to retrieve, send data to, update or delete.
- **Headers**: Additional information about the request or response, like the content type of the body.
- **Body**: The body of the request, used when sending data in POST or PUT requests.

One way to think about an HTTP request is to imagine ordering food at a restaurant. The method is the type of action you want to take, such as asking to see the menu, placing an order, changing an order, or canceling it. The URL is like telling the server exactly which restaurant counter to go to. The headers are like the extra details you give, such as saying you want vegetarian food or that you would like water with your meal. The body is the actual order information, such as the list of food you want to buy.

The method tells the server what kind of action the client wants to perform. For example, a GET request is used when the client wants to receive information from the server, such as loading a web page or asking for a list of items. A POST request is used when the client wants to send new information to the server, such as creating an account or submitting a form. A PUT request is used to update something that already exists, and a DELETE request is used to remove something.

The URL works like an address. It tells the request where to go. If the method explains what the client wants to do, the URL explains where that action should happen. For

example, if a web app wants a list of users, it might send a request to a URL that ends in `/users`. If it wants information for one specific user, it might send a request to a URL like `/users/5`.

Headers help the client and server understand each other better. They carry extra information along with the request or response, such as what type of data is being sent. This is similar to writing "Fragile" on a package or adding a note that says "Please handle with care." The message is still the same package, but the note helps explain how it should be handled.

The body is the part of the request that contains the actual data being sent. For example, when a user fills out a sign-up form, the name, email address, and password they enter may be placed in the body of the request. Not every request needs a body. GET requests usually do not send one, because they are mainly asking to receive information rather than send it.

1.2 Using HTTP with Web-Based APIs

Software developers often use HTTP to interact with web-based APIs. An Application Programming Interface (API) is a set of rules that allows programs to talk to each other. The developer creates the API on the server and allows the client to talk to it.

When a developer makes a request to an API endpoint, they're asking the server to either send them some data or receive some data from them. The response from the server will often be in a format like JSON or XML, which the developer can then use in their own application.

For example, a developer might make a GET request to `https://api.example.com/users` to retrieve a list of all users. The server would respond with a list of users in a format like JSON.

An API endpoint is a specific URL that a client can use to communicate with a server. Each endpoint usually has a clear purpose. One endpoint might return a list of books, while another might return information about a single book. In the same way that a restaurant has different counters for ordering food, paying, or picking up an order, an API can have different endpoints for different tasks.

When a client sends a request to an API, the server reads the request and decides how to respond. If the request is valid, the server sends back a response with the requested data or a message saying the action worked. If something is wrong, the server sends back a response explaining that there was a problem. This back-and-forth communication is one of the main ways web applications work behind the scenes.

For example, imagine a school website that stores a list of students. A client could send a GET request to an endpoint like `/students` to ask for the full list. It could send a

POST request to the same endpoint to add a new student. It could send a PUT request to `/students/3` to update the information for one student, or a DELETE request to `/students/3` to remove that student from the list. These requests follow the same HTTP ideas, even if the website or app itself looks very different on the screen.

Many modern apps work this way. A weather app may send a request to get the latest forecast. A music app may send a request to get a list of songs. A shopping app may send a request to place an order. Even though users may only see buttons, menus, and pictures, HTTP requests are often moving data in the background.

1.3 HTTP Responses

After a server receives an HTTP request, it sends back an HTTP response. The response is the server's answer to the client. Just as a request has several parts, a response also has its own important parts.

An HTTP response usually includes a status code, headers, and sometimes a body. The status code tells the client what happened. The headers give extra information about the response. The body contains the returned data, if there is any.

A simple way to think about a response is to imagine asking a librarian for a book. The librarian might say, "Here is the book you asked for," "That book is checked out," or "We do not have that book." These are different kinds of responses to the same kind of request.

Some common HTTP status codes are:

- 200: The request worked, and the server is sending back the requested data.
- 201: The request worked, and the server created something new.
- 400: The request had a problem, such as missing or incorrect information.
- 404: The server could not find what was requested.
- 500: Something went wrong on the server.

These codes help the client understand the result quickly. Instead of reading a long message first, the client can look at the status code and get a fast summary of what happened.

1.4 Data Formats in Responses

When a server sends data back to a client, that data needs to be organized in a way both sides can understand. Two common formats are JSON and XML. These formats are simply ways of arranging information so that programs can read it clearly and consistently.

JSON is one of the most common formats used with web APIs. It is popular because it is lightweight and easy for both humans and programs to read. A JSON response might include data written as key–value pairs. For example, a user might be described with data such as a name, age, and email address.

Here is a small example of JSON:

```
{
  "name": "Ava",
  "grade": 10,
  "club": "Robotics"
}
```

In this example, `"name"`, `"grade"`, and `"club"` are the labels, and `"Ava"`, `10`, and `"Robotics"` are the values. This is similar to filling out an information card. Each label tells what kind of information is being stored, and each value is the actual information.

XML is another format that may be used. It organizes information with opening and closing tags. While it is less common in many modern beginner examples, it is still useful to know that different APIs may return data in different formats.

CHAPTER 2

Using and Creating APIs

As a software engineer, you are likely familiar with building applications that interact with various external services and data sources. One of the most common methods for communication and integration is through HTTP APIs (Application Programming Interfaces). HTTP APIs provide a standardized way for applications to exchange data and functionality over the internet.

This chapter will introduce you to the world of HTTP APIs and explore how you can leverage them in your software development projects. We will cover the fundamental concepts, techniques, and best practices for effectively working with HTTP APIs.

An HTTP API allows two software systems to communicate and exchange information using the Hypertext Transfer Protocol (HTTP). It enables your application to make requests to an API server and receive responses in a structured format, such as JSON (JavaScript Object Notation) or XML (eXtensible Markup Language).

Using HTTP APIs offers a range of benefits for software engineers. It allows you to leverage external services and data sources, enabling your application to access functionality or retrieve valuable information from third-party systems. This opens up opportunities for integration with popular platforms, social media networks, payment gateways, geolocation services, and much more.

Throughout this chapter, we will explore various aspects of working with HTTP APIs, including:

- API endpoints and methods: Understanding how to interact with an API involves identifying the available endpoints and the supported methods, such as GET, POST, PUT, DELETE, and so on. We will discuss how to construct API requests and handle different response formats.
- Authentication and authorization: Many APIs require authentication to ensure secure access and protect sensitive data. We will delve into different authentication mechanisms, including API keys, tokens, OAuth, and other authentication protocols commonly used in API integrations.
- Request parameters and payloads: APIs often accept additional parameters or payloads to customize the request or send data for processing. We will explore how to pass query parameters, request headers, and request bodies when interacting with APIs.
- Error handling and status codes: Learning how to handle errors and interpret status

codes returned by APIs is crucial for building robust and resilient applications. We will discuss common status codes and best practices for handling various scenarios gracefully.

- Rate limiting and throttling: Many APIs impose restrictions on the number of requests you can make within a given timeframe to prevent abuse and ensure fair usage. We will cover techniques for handling rate limiting and implementing efficient strategies to manage API quotas.
- API documentation and testing: Proper documentation is essential for understanding an API's capabilities, endpoints, and expected behavior. We will explore how to read and interpret API documentation, as well as techniques for testing and validating API integrations.

By mastering the art of using HTTP APIs, you will expand your development toolkit and gain the ability to seamlessly integrate your applications with external services, leverage their functionalities, and build powerful, interconnected systems.

So, let's dive into the world of HTTP APIs and uncover the endless possibilities they offer for enhancing your software engineering projects.

2.1 Setting up an API

Create a file called `server.js`. This will be a basic webserver that we will use to test our an API. This is called a *RESTful API*. Write the following code to create a basic server for our API.

```
const express = require("express");

const app = express();
const PORT = 3000;

const student = {
  student_id: 42,
  name: "John Smith",
  major: "Computer Science",
  credits_completed: 88
};

app.get("/students/42", (req, res) => {
  res.json(student);
});

app.listen(PORT, () => {
  console.log(`API running at http://localhost:${PORT}`);
});
```

Then, run the following

```
npm init -y
npm install express
node server.js
```

This will create a new server, along with a ton of dependency files that javascript and node automatically install as part of your server.

2.1.1 What is an endpoint?

An *endpoint* is a URL exposed by the API. It usually is able to display a page or data based on the request of the endpoint.

We can navigate to the endpoint `GET /students/42`, where students is the resource that usually maps to a database, and 42 is the id of a student in the database. To retrieve this student from the database, we can navigate to

```
GET http://localhost:3000/students/42
```

which is an endpoint from our API. This will output

```
{
  "student_id": 42,
  "name": "John Smith",
  "major": "Computer Science",
  "credits_completed": 88
}
```

Note that this is done in json format, common for backend data parsing.

Each endpoint combines a resource path with an HTTP method to complete an action.

This leads us to the idea of *CRUD operations*. CRUD, which stands for CREATE, REMOVE, UPDATE, DELETE, are a form of outlining an API's routes. Calling a route that aims to alter should follow one of the CRUD operations. GET does not have an operation since it is *read-only*.

For a database representing students (assume more than just one), we would want to have routes that can complete each CRUD operation.

Before an API can receive data from a client, Express must parse JSON request bodies.

HTTP Method	Purpose	CRUD Op	Example
GET	Fetch Data	N/A	Get student information
PUT	Update Data	UPDATE	Fully update student
PATCH	Update section of Data	UPDATE	Update a student's major
POST	Create Data	CREATE	Create a new student
DELETE	Remove Data	DELETE	Remove a student after graduation

Add the following line after creating the application:

```
app.use(express.json());
```

This middleware automatically converts incoming JSON into a JavaScript object that can be accessed through `req.body`.

2.1.2 Reading all resources

Most APIs provide a route for retrieving an entire collection. Change the students array to look like the following.

```
const students = [
  {
    student_id: 42,
    name: "John Smith",
    major: "Computer Science",
    credits_completed: 88
  },
  {
    student_id: 43,
    name: "Jane Doe",
    major: "Mathematics",
    credits_completed: 61
  }
];

app.get("/students", (req, res) => {
  res.json(students);
});
```

The endpoint

```
GET /students
```

returns every student currently stored in the collection. Notice that we did not include an id. Now, there are more than one possible ids, so we need to make a general route that takes in an id of a student to fetch

```
app.get("/students/:id", (req, res) => {
  // use the id that is passed in the route
  const id = Number(req.params.id);
  // find a student from our students array
  const student = students.find((s) => s.student_id === id);

  // if the id is not found, we assume the student is not in the array
  if (!student) {
    return res.status(404).json({ error: "Student not found" });
  }

  res.json(student);
});
```

Start the server. Now, `http://localhost:3000/students` returns the entire students array, while `http://localhost:3000/students/43` only returns information about Jane Doe.

Routing to `http://localhost:3000/students/19` is invalid, so it returns

```
{"error":"Student not found"}
```

2.2 Requests with cURL

Curl, sometimes stylized as `cURL` or `curl`, is a command-line based resource used for sending API and HTTP requests for web server data.

The simplest curl command

```
curl http://example.com
```

which will return the `html` source code of the website `http://example.com`.

```
<!doctype html>
<html lang="en">

<head>
    <title>Example Domain</title>
    <link rel="icon" href="data:,">
    <meta name="viewport" content="width=device-width, initial-scale=1">
```

```
    <style>
        body {
            background: #eee;
            width: 60vw;
            margin: 15vh auto;
            font-family: system-ui, sans-serif
        }

        h1 {
            font-size: 1.5em
        }

        div {
            opacity: 0.8
        }

        a:link,
        a:visited {
            color: #348
        }
    </style>
</head>

<body>
    <div>
        <h1>Example Domain</h1>
        <p>This domain is for use in documentation examples without needing
        ↪ permission. Avoid use in operations.</p>
        <p><a href="https://iana.org/domains/example">Learn more</a></p>
    </div>
</body>

</html>
```

2.3 Updating and Deleting Resources with cURL

So far our API can read students and create new ones. To finish out the CRUD table from earlier, we need routes for `PUT`, `PATCH`, and `DELETE`, and we need to know how to trigger each of them with curl.

2.3.1 Fully replacing a resource with PUT

A `PUT` request means *replace this entire resource with what I'm sending you.* If you leave out a field, that field is expected to disappear or reset, because the whole object is being overwritten.

```
app.put("/students/:id", (req, res) => {
  const id = Number(req.params.id);
  const index = students.findIndex((s) => s.student_id === id);

  if (index === -1) {
    return res.status(404).json({ error: "Student not found" });
  }

  // PUT replaces the whole object, so we overwrite it completely
  students[index] = {
    student_id: id, ...req.body // this format keeps the same student_id and
    ↪  updates the rest of the fields with the new data
  };
  res.json(students[index]);
});
```

```
curl -X PUT http://localhost:3000/students/43 \
  -H "Content-Type: application/json" \
  -d '{
      "name": "Jane Doe",
      "major": "Physics",
      "credits_completed": 64
    }'
```

Notice we did not send `student_id` in the body – the route pulls it from the URL instead. Try leaving out `credits_completed` entirely and re-running the request. Because PUT replaces the whole object, that field will not be included in the response. This is the behavior that distinguishes PUT from PATCH.

2.3.2 Partially updating a resource with PATCH

A `PATCH` request means *only change the fields I'm sending you, and keep everything else the same.* This is almost always what you want when, say, updating a single field like a student's major.

```
app.patch("/students/:id", (req, res) => {
  const id = Number(req.params.id);
  const student = students.find((s) => s.student_id === id);

  if (!student) {
    return res.status(404).json({ error: "Student not found" });
  }

  // PATCH merges the incoming fields onto the existing object
  Object.assign(student, req.body);
```

```
  res.json(student);
});
```

```
curl -X PATCH http://localhost:3000/students/43 \
  -H "Content-Type: application/json" \
  -d '{ "major": "Physics" }'
```

Run a `GET` on `/students/43` afterward and confirm that `name` and `credits_completed` are untouched; only `major` changed. That single-field behavior is the whole point of PATCH.

2.3.3 Removing a resource with DELETE

A `DELETE` request removes a resource. Unlike POST, PUT, and PATCH, it typically does not need a request body; the URL alone tells the server what to remove.

```
app.delete("/students/:id", (req, res) => {
  const id = Number(req.params.id);
  const index = students.findIndex((s) => s.student_id === id);

  if (index === -1) {
    return res.status(404).json({ error: "Student not found" });
  }

  students.splice(index, 1);
  // status 204 means "success, but there is nothing to send back"
  res.status(204).send();
});
```

```
curl -X DELETE http://localhost:3000/students/44
```

Run `GET /students` again and Jerry Doe should be gone. If you run the same `DELETE` command a second time, you should get a `404`, since the student no longer exists to delete.

2.3.4 Headers with `-i` and `-v`

Every curl command so far has only shown you the body of the response. But remember: every HTTP exchange also includes headers and a status code, even when curl hides them by default. Two flags make that conversation visible.

`-i` includes the response headers above the body:

```
curl -i http://localhost:3000/students/43
```

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 78

{"student_id":43,"name":"Jane Doe","major":"Physics","credits_completed":64}
```

`-v` (verbose) goes further and shows the *request* curl sent, not just the response it received – useful when a request is failing and you aren't sure what curl is actually sending:

```
curl -v -X PATCH http://localhost:3000/students/43 \
  -H "Content-Type: application/json" \
  -d '{ "major": "Physics" }'
```

Reading a `-v` output line by line is one of the best habits a beginner can build: lines starting with `>` are what curl sent, lines starting with `<` are what the server sent back.

2.4 Authentication and API Keys

Every route we've built so far is completely open: anyone who knows the URL can read, create, update, or delete students. Real APIs usually can't work that way, since that data might be private, or changing it might cost money (imagine n API that sends text messages or charges a credit card). To restrict access, APIs commonly require an *API key*: a long, unique string that identifies who is making the request.

An API key is typically sent as a request header, most often `Authorization` or a custom header like `X-API-Key`:

```
curl http://localhost:3000/students \
  -H "Authorization: Bearer abc123yourkeyhere"
```

On the server, you check for that header before doing anything else:

```
const API_KEY = "abc123yourkeyhere";

function requireApiKey(req, res, next) {
  const authHeader = req.headers.authorization;
```

```
5
6     if (authHeader !== `Bearer ${API_KEY}`) {
7       return res.status(401).json({ error: "Unauthorized" });
8     }
9
10    next();
11  }
12
13  app.get("/students", requireApiKey, (req, res) => {
14    res.json(students);
15  });
```

If the header is missing or wrong, the client gets back a `401 Unauthorized` instead of the data.

A few rules are worth internalizing early, because they matter far beyond this textbook:

- **Never commit an API key to a public repository.** Once a key is pushed to code repository such as GitHub, assume it is compromised, even if you delete it in a later commit. If you do accidentally commit a key, revoke it immediately and generate a new one.
- **Never hardcode a real key directly in your source code.** The example does for simplicity. Real projects store keys in environment variables (often loaded from a `.env` file that is excluded from version control or public repositories) and read them with something like `process.env.API_KEY`.
- **Treat an API key like a password.** Anyone who has your key can act as you. They could be reading your data, spending your quota, or in some APIs, spending your money.
- **You can only access API's with their specific key.** If you have multiple API's, each one will have its own key. You cannot use the same key for different API's. For example, Spotify's public API has its own key, and a method for you to generate your own key. You cannot use a key from another API (such as OpenAI) to access Spotify's API. The strength here is that if one key is compromised, it does not compromise all of your API's. Secondly, this way, if a breach of data is attempted, Spotify knows who owns the key and can revoke it. Feel free to learn more about Spotify's API and how to generate your own key at https://developer.spotify.com/documentation/web-api/. Becoming increasingly more common is OpenAI's API, which has its own key and method for generating your own key. You can learn more about OpenAI's API at https://platform.openai.com/docs/guides/authentication.

This is the difference between *authentication* (proving *who* you are, usually with a key or token) and *authorization* (determining *what* you're allowed to do once you've proven who you are). A key can identify you correctly and still be denied access to a specific route if

you don't have permission for it.

2.5 Creating resources with POST

That's all great, but we can put cURL to work on much more laborious tasks. Going back to our created API example, we can post, update, and delete resources using cURL.

A POST request creates a new resource. Unlike GET requests, POST requests send data inside the request body.

```
app.post("/students", (req, res) => {
  const student = req.body;

  // basic validation: ensure a body is received
  if (!student) {
    return res.status(400).json({ error: "Invalid student data" });
  }

  students.push(student);

  // status 201 means that the student was pushed successfully
  res.status(201).json(student);
});
```

Notice the use of

```
req.body
```

which contains the JSON sent by the client.

HTTP requests are composed of headers and a body. Headers are metadata such as `Content-Type` (most commonly application/json) or `Authorization` that describe how the request should be handled. The body is the payload that carries actual data, like the student record being created by a POST request.

In Express, request headers are available on `req.headers`. For JSON payloads, middleware like `express.json()` parses the body so that the data is available on `req.body`.

A template for sending json in the body looks something like the following:

```
curl -X METHOD URL \
     -H "Header: Value" \
     -d 'JSON DATA'
```

To send a request body, include the body data in the HTTP request and set the appropriate `Content-Type` header. For example, with curl:

```
curl -X POST http://localhost:3000/students \
  -H "Content-Type: application/json" \
  -d '{
      "student_id": 44,
      "name": "Jerry Doe",
      "major": "Computer Science",
      "credits_completed": 23
    }'
```

This can be broken down as:

- `curl`: The command-line tool for making HTTP requests.
- `-X POST`: Use the POST HTTP method.
- `http://localhost:3000/students`: The endpoint being contacted. Must exist in our server routes.
- `-H "Content-Type: application/json"`: Tell the server the request body contains JSON.
- `-d '{ ... }'`: Send this JSON as the request body.

The request body in this example is the JSON string passed with `-d`. Express will parse that JSON into `req.body` when `express.json()` middleware is enabled.

Note that our server.js students array will not update with Jerry Doe, because the server only stores the students array upon initialization, but we can see our now three students by running a GET request on `/students`

```
[
  {
    "student_id": 42,
    "name": "John Smith",
    "major": "Computer Science",
    "credits_completed": 88
  },
  {
    "student_id": 43,
    "name": "Jane Doe",
    "major": "Mathematics",
    "credits_completed": 61
  },
  {
```

cURL	Postman
The URL itself	The URL bar
`-X METHOD`	The method dropdown
`-H "Header: Value"`	The Headers tab
`-d '{...}'`	The Body tab (raw / JSON)
`-i` / `-v` output	The Response panel and Console

```
    "student_id": 44,
    "name": "Jerry Doe",
    "major": "Computer Science",
    "credits_completed": 23
  }
]
```

2.6 Postman

Typing curl commands by hand is a great way to *learn* how HTTP requests actually work, because every piece of the request (the method, the headers, the body) has to be spelled out yourself. But once you already understand what those pieces are, typing them out every single time gets tedious and error-prone, especially for requests with long JSON bodies or several headers.

Postman is a GUI application for building, sending, and saving HTTP requests. Everything you've been typing as curl flags has a matching box in Postman:

2.6.1 Building your first request in Postman

1. Open Postman (download available here) and click **New** → **HTTP Request**.
2. Set the method dropdown (left of the URL bar) to `GET`, and enter `http://localhost:3000/students` in the URL bar.
3. Click **Send**. The response body appears in the panel below, along with the status code and response time: the same information `-i` shows you in curl, just laid out visually instead of printed as text.

Now recreate the POST request from earlier:

1. Change the method to `POST` and set the URL to `http://localhost:3000/students`.
2. Open the **Body** tab, select **raw**, and choose **JSON** from the format dropdown on the right. Postman will automatically set the `Content-Type: application/json` header for you. This is one thing it does that curl makes you do by hand.

3. Paste in a student object:

```
{
  "student_id": 45,
  "name": "Alex Rivera",
  "major": "Biology",
  "credits_completed": 12
}
```

4. Click **Send** and confirm you get a `201 Created` status back with the same object.

2.6.2 Why this matters once requests get complicated

A single GET request is barely worth using Postman for; a browser address bar does the job just as well. Postman starts to pay off once your API involves things that are painful to keep retyping in a terminal:

- **Collections.** Every request you build can be saved into a named *collection*, so your entire API (every route in this chapter, for example) can be saved, organized, and rerun with a click instead of retyped.
- **Environments.** A variable like `{{base_url}}` can be swapped between `http://localhost:3000` and a real deployed server without editing every saved request individually.
- **Authorization tab.** Instead of manually adding an `Authorization` header on every request, you can set your API key once per collection and Postman attaches it automatically.
- **Automated testing**: Postman can run a series of requests and check that the responses match expected values, so you can verify your API is working as intended after every change. This is great for testing before you have a front-end built, or for running regression tests after a code change.

Neither tool replaces the other. cURL is scriptable and works anywhere you have a terminal, including inside automated tests and shell scripts, which is why you'll keep seeing it later in this book. Postman is better suited to exploring an API interactively, organizing a large set of requests, and sharing that organized set with teammates. Most professional API developers end up using both, for different parts of their workflow.

2.7 Summary

This chapter touched on a lot of software and development concepts. Here are some of the key takeaways:

- API's are ways to connect and fetch data from a server. They are a way to connect two different applications together.
- API's are usually built with a RESTful architecture, which is a set of principles that follow HTTP protocol to perform CRUD operations on resources.
- **CURL** is a command line tool that allows you to send HTTP requests to an API. Another commonly used tool is **Postman**, which is a GUI application that allows you to send HTTP requests to an API.
- We created a basic API that allows us to perform CRUD operations on a student resource. We can create, read, update, and delete students from our API.

Chapter 3

Data Compression and Decompression

Data compression and decompression are fundamental techniques used in modern computing, enabling efficient storage and transmission of data. The concept of entropy, borrowed from the field of information theory, plays a crucial role in determining the compression rate.

3.1 Data Compression and Decompression

Data compression is the process of reducing the amount of data needed to represent a particular set of information. The two main types of data compression are lossless and lossy. Lossless compression ensures that the original data can be perfectly reconstructed from the compressed data, whereas lossy compression allows some loss of data for more significant compression rates. Decompression is the reverse process of compression, reconstructing the original data from the compressed format.

3.2 Entropy

In information theory, entropy measures the unpredictability or randomness of information content. More specifically, it quantifies the expected value of the information contained in a message. Lower entropy implies less randomness and more repetitiveness, which in turn means the data can be compressed more.

3.3 Entropy and Compression

The role of entropy in data compression is fundamental. The entropy of a source of data is the minimum number of bits required, on average, to encode symbols drawn from the source. It serves as a lower bound on the best possible lossless compression rate.

For a source X with probability distribution $p(x)$, the entropy $H(X)$ is defined as:

$$H(X) = -\sum_{x \in X} p(x) \log_2 p(x) \tag{3.1}$$

If the entropy of the data is high (i.e., the data is random and unpredictable), the potential for compression is low. On the other hand, if the entropy is low (the data is predictable), the data can be compressed to a smaller size.

Chapter 4

Dealing with JSON and XML

Appendix A

Answers to Exercises



Index

API key, 15
API keys, 15
authentication, 16
authorization, 16
Authorization header, 15

collection, 20
CRUD operations, 9

data compression, 23

endpoint, 9
entropy, 23

HTTP, 3, 7

json, 25

lossless, 23
lossy, 23

Postman, 19

read-only, 9
RESTful API, 8

Web APIs, 7

xml, 25